λNTS

실습 : FVAE

프로그래밍언어개론

$$
\begin{array}{l}
e ::= n \mid x \mid e + e \mid e - e \mid let\ x\ =\ e\ in\ e \mid \lambda x.e \mid e\ e \\
n \in \mathbb{Z} \\
x \in Var
\end{array}
$$

정민경 (jmink002@gmail.com)

FVAE (First-class Function)

$$
\begin{aligned}
&\boldsymbol{p} ::= \boldsymbol{d}\ \boldsymbol{e} \\
&\boldsymbol{d} ::= \mathit{def}\ \boldsymbol{x}\ \boldsymbol{x}\ =\ \boldsymbol{e} \\
&\boldsymbol{e} ::= n \mid \boldsymbol{x} \mid \boldsymbol{e} + \boldsymbol{e} \mid \boldsymbol{e} - \boldsymbol{e} \mid \mathit{let}\ \boldsymbol{x}\ =\ \boldsymbol{e}\ \mathit{in}\ \boldsymbol{e} \mid \boldsymbol{x}(\boldsymbol{e}) \\
&n \in \mathbb{Z} \\
&\boldsymbol{x} \in \mathit{Var}
\end{aligned}
$$

6주차 F1VAE (First-order Function)

$$
\begin{aligned}
&\boldsymbol{e} ::= n \mid \boldsymbol{x} \mid \boldsymbol{e} + \boldsymbol{e} \mid \boldsymbol{e} - \boldsymbol{e} \mid \mathit{let}\ \boldsymbol{x}\ =\ \boldsymbol{e}\ \mathit{in}\ \boldsymbol{e} \\
&n \in \mathbb{Z} \\
&\boldsymbol{x} \in \mathit{Var}
\end{aligned}
$$

5주차 VAE

$$
\begin{aligned}
&\boldsymbol{e} ::= n \mid \boldsymbol{x} \mid \boldsymbol{e} + \boldsymbol{e} \mid \boldsymbol{e} - \boldsymbol{e} \mid \mathit{let}\ \boldsymbol{x}\ =\ \boldsymbol{e}\ \mathit{in}\ \boldsymbol{e} \mid \lambda \boldsymbol{x}.\boldsymbol{e} \mid \boldsymbol{e}\ \boldsymbol{e} \\
&n \in \mathbb{Z} \\
&\boldsymbol{x} \in \mathit{Var}
\end{aligned}
$$

(이번주) 7주차 FVAE (First-class Function)

F1VAE vs FVAE

$$
\begin{aligned}
&\boldsymbol{p} ::= \boldsymbol{d}\ \boldsymbol{e} \\
&\boldsymbol{d} ::= \mathit{def}\ \boldsymbol{x}\ \boldsymbol{x}\ =\ \boldsymbol{e} \\
&\boldsymbol{e} ::= \boldsymbol{n} \mid \boldsymbol{x} \mid \boldsymbol{e}+\boldsymbol{e} \mid \boldsymbol{e}-\boldsymbol{e} \mid \mathit{let}\ \boldsymbol{x}\ =\ \boldsymbol{e}\ \mathit{in}\ \boldsymbol{e} \mid \boldsymbol{x}(\boldsymbol{e}) \\
&\boldsymbol{n} \in \mathbb{Z} \\
&\boldsymbol{x} \in \mathit{Var}
\end{aligned}
$$

6주차 F1VAE (First-order Function)

$$
\begin{aligned}
&\boldsymbol{e} ::= \boldsymbol{n} \mid \boldsymbol{x} \mid \boldsymbol{e}+\boldsymbol{e} \mid \boldsymbol{e}-\boldsymbol{e} \mid \mathit{let}\ \boldsymbol{x}\ =\ \boldsymbol{e}\ \mathit{in}\ \boldsymbol{e} \mid \lambda \boldsymbol{x}.\boldsymbol{e} \mid \boldsymbol{e}\ \boldsymbol{e} \\
&\boldsymbol{n} \in \mathbb{Z} \\
&\boldsymbol{x} \in \mathit{Var}
\end{aligned}
$$

(이번주) 7주차 FVAE (First-Class Function)

λNTS

λNTS

First-order function

함수를 변수와 다르게 특별 취급

- 함수를 인자로 전달하거나 반환할 수 없으므로, 함수의 이름과 변수의 이름을 명확하게 구분
  - Second-class citizen: 함수를 2등 시민 취급
- 함수와 변수의 사용처가 명확하게 다름
  - 별도의 메모리 공간에 함수와 변수를 따로 저장

  ⇒ *Store*외에 함수를 저장할 별도의 메모리가 필요

대부분의 imperative language(non-functional language)는 first-order function을 지원

- Functional language는 higher-order function을 지원

λNTS

First-class functions

First-class function을 지원하는 언어에서는 함수이름과 변수이름이 구별되지 않음

- 함수가 "값"으로 활용하여 변수에 저장
  - 함수를 변수에 저장
  - 함수를 함수의 인자로 전달
  - 함수를 반환
- Functional language는 기본적으로 first-class function을 지원
  - i.e. higher-order function을 지원

과제 : interp

- FVAE AST와 store를 입력으로 받아 해석한 후 그 결과를 반환하는 함수 interp을 아래와 같이 정의하시오.

```
(* hw7.ml *)
```

- interp: Ast.expr -> Store.t -> Store.value

과제 : interp

- First-class Function을 지원하는 FVAE에서는 함수도 값 (value)
- Store.value의 확장이 필요

```
type t = (string * value) list
and value =
  | NumV of int
  | ClosureV of string * Ast.expr * t
```

(* store.ml *)

과제 : interp

```
type expr =
  | Num of int
  | Id of string
  | Add of expr * expr
  | Sub of expr * expr
  | LetIn of string * expr * expr
  | App of expr * expr
  | Lambda of string * expr
```

(* parser/ast.ml *)

- FVAE AST의 정의는 아래와 같다
  - Num : 정수 literal을 나타내는 AST 노드
  - Id : 변수를 나타내는 AST 노드
  - Add : 두 expr 간의 덧셈 연산을 나타내는 AST 노드
  - Sub : 두 expr 간의 뺄셈 연산을 나타내는 AST 노드
  - LetIn : 변수에 expr 계산 값을 할당한 후, 다음 expr을 계산함을 나타내는 AST 노드
  - App : 함수 적용을 나타내는 AST 노드
  - Lambda: 이름없는 함수를 나타내는 AST 노드

과제 : interp

- interp : Ast.expr -> Store.t -> Store.value

```
type expr =
  | Num of int
  | Id of string
  | Add of expr * expr
  | Sub of expr * expr
  | LetIn of string * expr * expr
  | App of expr * expr
  | Lambda of string * expr
```

$n.$ $$\sigma \vdash n \Downarrow_E n$$

$x.$ $$\frac{x \in Domain(\sigma)}{\sigma \vdash x \Downarrow_E \sigma(x)}$$

$e_1 + e_2.$ $$\frac{\sigma \vdash e_1 \Downarrow_E n_1 \qquad \sigma \vdash e_2 \Downarrow_E n_2}{\sigma \vdash e_1 + e_2 \Downarrow_E n_1 +_{\mathbb{Z}} n_2}$$

$e_1 - e_2.$ $$\frac{\sigma \vdash e_1 \Downarrow_E n_1 \qquad \sigma \vdash e_2 \Downarrow_E n_2}{\sigma \vdash e_1 - e_2 \Downarrow_E n_1 -_{\mathbb{Z}} n_2}$$

$let\, x = e_1\, in\, e_2.$ $$\frac{\sigma \vdash e_1 \Downarrow_E v_1 \qquad \sigma[x \mapsto v_1] \vdash e_2 \Downarrow_E v_2}{\sigma \vdash let\, x = e_1\, in\, e_2 \Downarrow_E v_2}$$

$\lambda x.e.$ $$\sigma \vdash \lambda x.e \Downarrow_E < \lambda x.e, \sigma >$$

$e_1\, e_2.$ $$\frac{\sigma \vdash e_1 \Downarrow_E < \lambda x.e_3, \sigma_1 > \qquad \sigma \vdash e_2 \Downarrow_E v_1 \qquad \sigma_1 \vdash e_3[v_1/x] \Downarrow_E v_2}{\sigma \vdash e_1\, e_2 \Downarrow_E v_2}$$

과제 : interp

- 예외처리

```
type expr =
  | Num of int
  | Id of string
  | Add of expr * expr
  | Sub of expr * expr
  | LetIn of string * expr * expr
  | App of expr * expr
  | Lambda of string * expr
```

계산 결과 NumV가 아닌 경우 :
Not a number: <expr1> + <expr2>

failwith (Format.asprintf “Not a number: %a + %a” Ast.pp expr1 Ast.pp expr2)

(Sub인경우 연산자만 - 로 변경)

계산 결과 ClosureV가 아닌 경우 : Not a function: <expr>

failwith (Format.asprintf “Not a function: %a” Ast.pp expr)

과제 : interp

- 주어진 templete project의 구조는 다음과 같음

```
├── parser
│   ├── ast.ml
│   ├── dune
│   ├── lexer.mll
│   ├── parser.mly
│   └── parserMain.ml
```

- parser 사용 예시
  - e.g.

    ParserMain.parse “let x = 3 in x + 1”

    = Ast.LetIn (“x”, Ast.Num 3, Ast.Add (Ast.Id “x”, Ast.Num 1))

Test

- **hw7_test.ml**에 Store.find, Store.add, interp에 대해 테스트 각 3개씩 총 9개 이상 작성
- e.g.
  - let%test _ = Store.find "x" [(“x”, NumV 1)] = NumV 1
  - let%test _ = Store.find "f" [(“f”, ClosureV ("x", (Id "x"), []))] = ClosureV ("x", Id "x", [])
  - let%test _ = Store.add "x" (NumV 99) [] = [("x", NumV 99)]
  - let%test _ = Hw7.interp (ParserMain.parse "let x = 1 in x + 2") [] = NumV 3

실행 예시 및 참고1

Multiple parameters

interp (ParserMain.parse “(fun x y -> x + y) 1 2”) Store.empty

= Store.NumV 3

(* 참고 : 이론자료 Multiple parameters support as syntactic sugar *)

```
utop # ParserMain.parse "(fun x y -> x + y) 1 2";;
- : Ast.expr =
Ast.App
 (Ast.App
   (Ast.Lambda ("x", Ast.Lambda ("y", Ast.Add (Ast.Id "x", Ast.Id "y"))),
   Ast.Num 1),
 Ast.Num 2)
```

실행 예시 및 참고2

Named function

interp (ParserMain.parse “let f = (fun x y -> x + y) in f 1 2”) Store.empty
= Store.NumV 3

(* 참고 : 이론자료 Named function support as syntactic sugar *)

```
utop # ParserMain.parse "let f = (fun x y -> x + y) in f 1 2";;
- : Ast.expr =
Ast.LetIn ("f",
 Ast.Lambda ("x", Ast.Lambda ("y", Ast.Add (Ast.Id "x", Ast.Id "y"))),
 Ast.App (Ast.App (Ast.Id "f", Ast.Num 1), Ast.Num 2))
```

주의 사항

- 주어진 템플릿 모두 포함하여 제출
  - 구현 코드(hw7.ml, hw7_test.ml, store.ml)와 parser 폴더 모두 포함하여 제출

- 과제 함수 외에 다른 함수 필요 시 정의 가능
  - 다른 함수들 역시 hw7.ml or store.ml 파일 안에 함께 작성
  - **주어진 함수 타입은 반드시 준수! (함수 이름, 타입 오류로 인한 컴파일 에러 시 0점)**

- 과제 구현하며 필요한 라이브러리 함수 사용 가능
  - e.g., List library (https://ocaml.org/manual/5.3/api/List.html)

과제 제출

- 제출 기한 : **5월 1일 목요일 23:59:59 까지** (추가 제출 기한 없음)
- 제출 파일 : hw7.ml, store.ml, hw7_test.ml, dune, parser 폴더
- 제출 형식 : 하나의 파일로 압축 후 제출 (압축파일명 : 학번.zip)

- 주의 사항
  - 명시된 제출 파일 이외 다른 파일 제출 금지 (dune clean 후 제출)
  - 압축 해제 시, 제출 파일이 바로 등장하도록 함 (폴더 내에 포함시키지 말 것)
  - **파일명, 함수 이름, 파라미터 개수 및 순서** 반드시 준수
  - 대소문자, 띄어쓰기와 같은 오타 전부 예외없이 감점
  - 컴파일 에러 발생 시 0점
  - ChatGPT 사용 금지

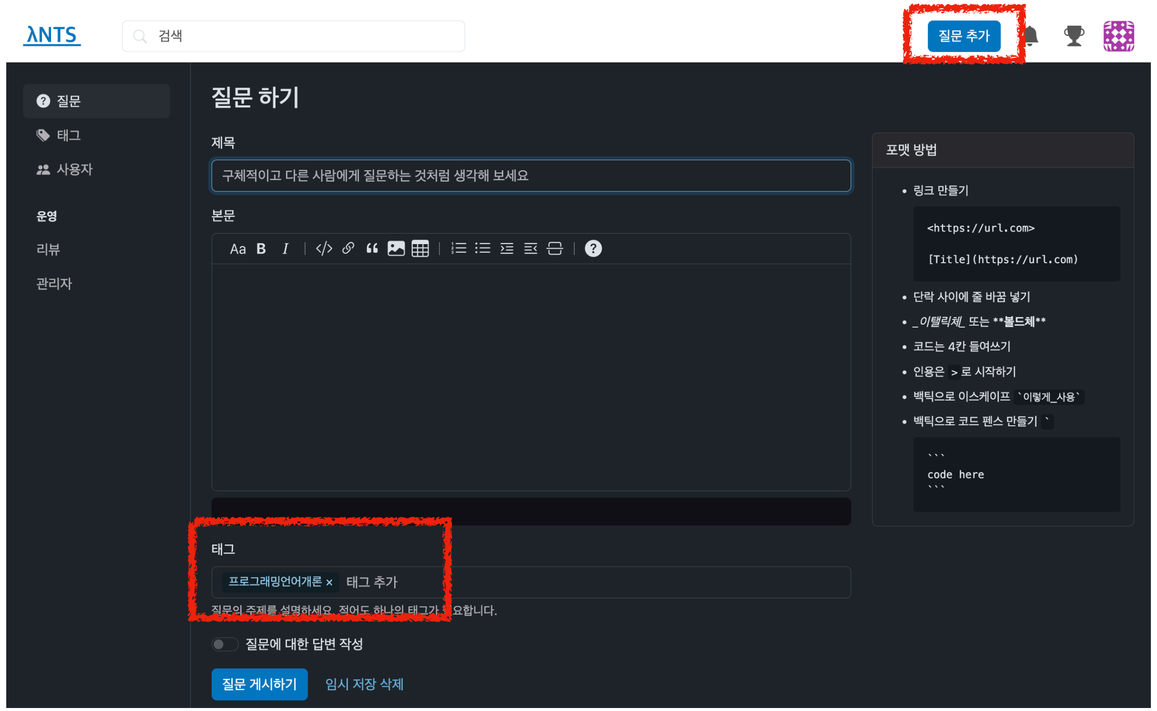
5주차 과제 점수 확인

- 점수에 이상이 있거나 궁금한 점이 있다면
- 매주 실습시간에 튜터에게 문의하시면 됩니다

- 실습시간 이후에는 과제 점수에 대한 문의, 클레임 받지 않습니다

Question?

- 수업 or 과제와 관련된 질문은 질문페이지을 통해 질문
  - https://answer.cnu-ants.synology.me/tags/upl
  - 프로그래밍언어개론 태그로 질문 추가
  - “N주차 질문”과 같은 모호한 제목 자제 (질문 내용을 한줄 요약하여 제목으로)
  - 질문을 공유합시다 :)



Question?

- 수업 or 과제가 아닌 개인적인 문의사항은 메일로
  - 00분반 TA : jmink002@gmail.com
  - 01분반 TA : sy.baek28@gmail.com
  - 메일 제목: [UPL25] 제목 + 이름